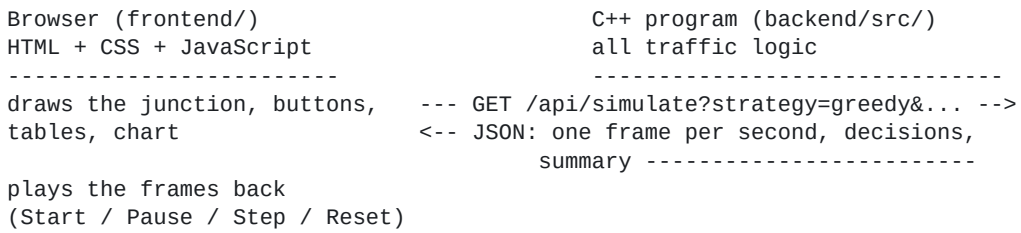
Traffic Signal Optimizer: C++ code guide

How the backend works, file by file, and how to explain it.

1. The big picture

The project has two halves that talk over HTTP on your own computer.



The C++ program does everything that matters: it creates the cars, runs the light, makes the decisions, and calculates the results. The browser never calculates traffic. It asks C++ for a complete 180-second run and then shows it one second at a time. That is why Start, Pause, Step and Reset are instant: they only move forward or back through frames C++ already sent.

The same C++ program also serves the web page, so there is only one thing to run.

2. What "Optimize signals" does

There are two ways to control the light:

- **Fixed-time** (the baseline): North-South gets 30 s, then East-West gets 30 s, forever. It never looks at the queues. This is how a basic old signal works.
- **Greedy + priority queue** (the optimizer): every time a green ends, it looks at how many cars are waiting, how many are expected, and how long each road has been red, then decides who goes next and for how long.

Pressing **Optimize signals** does three things: it switches the controller from Fixed-time to Greedy, it rewinds to 0 s so the optimized run gets exactly the same cars as the baseline (fair comparison), and it shows the first decision the optimizer will make, with its reasoning. It does not change the traffic, only who is controlling the light. Then you press Start and compare the two runs in the "Baseline vs optimized" table.

3. Reading order

Read the files in this order. Each one only uses the files above it.

File	What it does	DAA concept
model.hpp	Data types: directions, phases, Timing, Scenario, Approach, Signal, Decision	Data modelling
random.hpp	Seeded random numbers, Poisson arrivals, FNV hash	Determinism
scenarios.hpp	The 2 presets and the rules for valid settings	Input validation
arrivals.hpp	Builds every car arrival before the run starts	Pre-computation
priority_queue.hpp	Hand-written binary max-heap	Priority queue
segment_tree.hpp	Range sum / range max over time	Segment tree
controllers.hpp	Fixed-time and Greedy controllers	Greedy algorithm
engine.hpp	The second-by-second simulation and the light state machine	Simulation

metrics.hpp	Average wait, served, residual queue	
json.hpp, http_server.hpp, main.cpp	Turn results into JSON and serve them	Plumbing

4. The data (model.hpp)

Four **approaches** (roads into the junction): N, S, E, W, stored as array indexes 0 to 3. Two **phases**: NS (North and South move together) and EW. They never have green at the same time. The light is in one of three states: Green, Yellow or AllRed.

```
struct Approach {
    int queue;            // cars waiting now
    double arrivalRate;   // average cars arriving per second
    double serviceRate;   // cars that can leave per second of green
    long long accumulatedWait;
    int maxQueue, served, arrived;
    int redTime;          // seconds since this road last had green
    double credit;        // fractional capacity (for rates like 0.5 car/s)
};
```

5. Making the cars (random.hpp, arrivals.hpp)

Each second, each road gets a random number of cars from a **Poisson distribution** whose average is the arrival rate (for North, 0.35 cars per second). Real random numbers would make every run different, so a **seeded generator** (mulberry32) is used: seed 42 always gives exactly the same cars.

The whole run is generated **before** it starts, into `schedule[t][d]` = cars arriving on road d in second t. Both strategies receive this same schedule. This is the key to a fair comparison: any difference in results comes from the controller, not from luck.

Each road has its own random stream (seed XOR hash of the road name), so changing North's rate does not change East's cars.

6. One second of simulation (engine.hpp, step())

```
bool step() {
    // 1. arrivals: cars for this second join their queues
    for each road d:  queue[d] += schedule[t][d]

    // 2. service: roads with green let cars leave
    for each road d:
        if (light is Green and phase serves d) {
            credit += serviceRate;
            leave = min(queue, floor(credit));   // whole cars, never more than are there
            queue -= leave;  credit -= leave;  redTime = 0;
        } else { redTime++; }

    // 3. waiting: every car still in a queue adds 1 second of delay
        totalWait += queue;

    // 4. the light's clock
    time++;  advanceSignal();
}
```

advanceSignal() is a small state machine:

```
 Green --(time up)--> ask controller: same phase or other?
    same phase  -> stay Green for the extra seconds ("extend")
    other phase -> Yellow (3 s) -> AllRed (2 s) -> Green of the other phase
```

Safety rules are checked by the engine itself, whatever the controller says: a green is always clamped between minGreen (10 s) and maxGreen (60 s), and if a phase has had 60 s the engine forces a switch.

The two phases can never be green together, and every switch passes through yellow and all-red.

7. The baseline (FixedTimeController)

```
Decision decide(view) {
    d.phase    = otherPhase(view.phase);     // just alternate
    d.duration = fixedGreen[d.phase];        // always 30 s
}
```

8. The optimizer (GreedyController)

It runs every time a green ends, in three steps.

Step 1: score every road

```
score = alpha * cars waiting + beta * cars expected in a cycle + gamma * seconds on red
      =   1   * queue        +  1   * (rate * 100 s)          +  0.2  * redTime
```

A phase's priority is the sum of its two roads' scores. The third term (time on red) stops a road from being ignored just because its queue is short.

Step 2: priority queue picks WHO goes next

Both phases are pushed into the **max-heap**; the one with the highest priority comes out first. Three rules:

- A road red for 60 s or more (maxWait) makes its phase **overdue**: it gets a +1,000,000 bonus, so it always comes out first. This prevents starvation.
- A phase that has already been green for 60 s (maxGreen) is not pushed at all.
- If two priorities are equal, North-South wins (fixed tie-break, so results are repeatable).

Step 3: greedy split decides HOW LONG

The cycle has a green budget of 90 s. Each phase first gets the 10 s minimum. The other 70 s are handed out **one second at a time**: each second goes to the phase that would let the most still-waiting cars through in that second. It stops when the budget is used, when a phase hits 60 s, or early if no one needs more time.

```
alloc[NS] = alloc[EW] = minGreen;
while (used < budget) {
    best = phase with the largest gain, among phases below maxGreen
    // gain(p) = cars that one more second of green would let through:
    //   sum over p's roads of min(serviceRate, max(0, demand - serviceRate*alloc[p]))
    if (no phase gains anything) break;
    alloc[best]++;  used++;
}
```

This is a **greedy algorithm**: at every step it makes the choice that looks best right now (the most cars served by one more second) and never goes back to undo it.

Worked example (Peak demo, decision at 25 s)

North had green for its opening 25 s. One cycle = 90 s budget + 2 x 5 s transitions = 100 s.

Road	Queue	Expected (rate x 100)	Red for	Score
North	64	35	0 s	64 + 35 + 0 = 99
South	0	15	0 s	0 + 15 + 0 = 15
East	63	25	25 s	63 + 25 + 5 = 93
West	15	10	25 s	15 + 10 + 5 = 30

Priorities: North-South = 99 + 15 = **114**, East-West = 93 + 30 = **123**. The heap pops **East-West** first.

Greedy split (demand = queue + expected: N 99, S 15, E 88, W 25): both start at 10 s and each extra second is worth 2 cars to either side, so ties go to North-South until South runs out at 15 s. Then East-West is worth more (2 vs 1) until West runs out at 25 s. Then both are worth 1, North-South wins the ties up to its 60 s cap, and the last 5 s go to East-West. Result: **North-South 60 s, East-West 30 s**.

Decision: **switch to East-West for 30 s**. That is exactly what the page shows after pressing Optimize signals.

9. The priority queue (priority_queue.hpp)

A binary heap stored in a vector. The parent of index i is (i-1)/2; its children are 2i+1 and 2i+2. The largest item is always at index 0.

```
push(x): put x at the end, then swap it up while it beats its parent       O(log n)
pop():   take index 0, move the last item to index 0, swap it down
         with its better child until neither child beats it               O(log n)
```

Here there are only 2 phases, so the heap is small, but it is a real heap and the same code would schedule any number of phases (for example 4 independent approaches, or many intersections).

10. The segment tree (segment_tree.hpp)

While the run records, two segment trees are updated every second: a **sum tree** of cars arriving and a **max tree** of total queue length. The "Look up a time window" box asks C++ questions like "how many cars arrived between 10 s and 40 s?" and "what was the worst total queue in that window?".

A plain loop would take O(n) per question. The tree stores pre-combined values for blocks of time, so update and query are both **O(log n)**. It is stored as an array of size 2n: leaves at n...2n-1, each parent i combines children 2i and 2i+1.

11. Results (metrics.hpp)

- **Average wait** = total car-seconds of waiting / cars served. If nobody was served it is N/A, never a divide-by-zero.
- **Vehicles served**: cars that got through. **Residual queue**: cars still waiting at the end.
- **Longest queue**, **peak total queue**, **phase switches**.

12. Complexity summary

Part	Cost	Notes
Generate arrivals	O(T x 4)	T = seconds in the run
One simulation step	O(1)	plus O(log T) for the two tree updates
Heap push / pop	O(log P)	P = phases (2 here)
Greedy split	O(B x P)	B = budget seconds (70 here)
Whole run	O(T log T + D x B x P)	D = decisions (about 5)
Window query	O(log T)	per question

13. The server part (main.cpp, http_server.hpp, json.hpp)

Not traffic logic, just plumbing. **http_server.hpp** opens a socket on port 8080 (Winsock on Windows, POSIX sockets elsewhere), reads one request, calls a handler and sends the reply. **main.cpp** has three routes:

URL	Returns
/api/scenarios	The 2 presets and the default timing
/api/simulate?strategy=greedy&seed;=42&rN;=0.35&...	Full run: 181 frames, every decision with its explanation, final summary. Or a list of errors if a setting is invalid.
/api/query?from=10&to;=40&...	Segment-tree answers for that window
anything else	Files from the frontend folder (the web page)

json.hpp is a 70-line JSON writer, so no external library is needed.

14. Questions your teacher might ask

Why not just use std::priority_queue?

It would work. The heap is written by hand to show the algorithm (sift-up and sift-down) instead of hiding it in the standard library.

Is the greedy result optimal?

No. Greedy makes the best local choice each second and never reconsiders. It is fast and usually good, but not guaranteed optimal. The results table shows the real outcome, which can sometimes be worse than the baseline on some metric.

How do you make the comparison fair?

Both runs use one pre-generated arrival schedule from the same seed, the same starting queues, the same timing rules and the same metric formulas. The page shows the schedule ID to prove it.

Can the controller cheat by seeing future cars?

No. It only receives a ControllerView: current queues, time on red and average rates. The schedule is never passed to it.

How is starvation prevented?

Three ways: the gamma x redTime term raises a waiting road's score; a road red for 60 s is forced to the top of the heap; and maxGreen forces a switch, so every road gets green within 60 + 2 x 5 = 70 s.

Why does Peak demo only improve about 4%?

Demand there is close to the junction's total capacity, so there is little spare green time to redistribute. In Single-approach congestion, where one road is overloaded and the others are light, average wait halves (91.3 s to 44.9 s).

Where is the segment tree used?

For time-window questions over recorded history (sum of arrivals, max of queue), in O(log n). It is not needed for the queues themselves.

Is this safe for real signals?

No. It is a simplified educational model: no turning lanes, pedestrians, start-up delay or real sensors.

15. Build and run

```
Windows (MinGW g++), from the project folder:
  g++ -std=c++17 -O2 -o server.exe backend/src/main.cpp -lws2_32
  g++ -std=c++17 -O2 -I backend/src -o tests.exe backend/tests/tests.cpp
  server.exe              then open http://localhost:8080
  tests.exe               19 tests, 62 checks

Linux / macOS:
  make        ./server        ./tests
```